

Project 1: Simple Linux Shell Interface

Project Overview

The project, titled "Simple Linux Shell Interface," is about creating a basic Linux shell that allows users to execute fundamental Linux commands. This shell serves as an interface for users to interact with the operating system through command-line inputs. The target audience includes Linux users who require a straightforward shell for performing routine tasks and executing commands.

Solution Approach

I approached the project development in the following phases:

1. **Planning:** I started by reading the project objectives, figuring out what I wanted my shell to achieve based on the project requirements. I listed down the requirements and functionalities I expected from it, setting a clear goal.
2. **Research:** Before jumping into coding for this project, I took some time to study how the Linux command-line interface operates. I delved into shell scripting and explored various existing shell implementations to gather insights and design principles.
3. **Implementation:** With a clear plan, I started writing the code. I developed functions to handle user input prompt, command parsing, and execution. Special attention was given to implementing features like handling specific commands such as '/proc' to retrieve system information and adding several helper functions. I aimed for a modular code structure to make the codebase more manageable and understandable.
4. **Testing and Debugging:** Testing was a crucial part of the process. I conducted unit tests to ensure that each function worked correctly on its own. Integration testing was also carried out to verify how different components of the shell interacted with each other. I spent time using Valgrind and debugging issues related to command parsing, execution, etc.

5. Documentation: In order to ensure that others could easily understand and use my shell, I documented the code extensively. I wrote comments within the code to explain its functionality and created a README file providing an overview of the project and outlining the solution approach.

Challenges and Solutions

1. Memory Leaks: There were memory leaks that caused errors and performance issues. **Solution:** I addressed these issues by using Valgrind to detect leaks, reviewing and debugging code for proper memory management, and ensuring memory cleanup in all scenarios.

2. Testing: Testing the shell thoroughly to identify and fix bugs was challenging. **Solution:** I developed comprehensive test cases covering various scenarios and conducted thorough testing to ensure everything worked correctly and reliably.

3. 'history' Command: I had trouble implementing the 'history' command functionality. When displaying the last ten commands, only the first part of multi-part commands was shown. **Solution:** I revised the parsing logic to correctly capture and display the full command entered.

Testing and Validation

Testing Methodology:

1. Specific Commands: Users should be aware of special commands such as:

- 'ls' to list all files in the current directory (excluding hidden files).
- '/proc' for accessing system information.
- 'exit' for terminating the shell.

(Note: The hidden file '.421sh' was created in the home directory: '/home/minh/.421sh')

2. Unit Testing: Each function within the shell program was individually tested to ensure proper functionality.

- `get_user_command()`: Tested input retrieval including empty and special character cases.

- `parse_command()`: Tested correct tokenization and handling of commands with arguments.
- `execute_command()`: Tested successful execution and proper error handling.

Example Session

`./my_shell`

`$ ls -l`

`total 93`

`-rw----- 1 mle6 rpc 46088 Mar 10 20:25 '$'`

`-rwx----- 1 mle6 rpc 25056 Mar 10 20:22 my_shell`

`-rw-r--r-- 1 mle6 rpc 10705 Mar 10 19:59 my_shell.c`

`-rw-r--r-- 1 mle6 rpc 7981 Mar 7 14:49 utils.c`

`-rw-r--r-- 1 mle6 rpc 2388 Mar 7 14:50 utils.h`

`$ echo hello world`

`hello world`

`$ /proc /loadavg`

`1.12 1.26 1.44 3/724 1199497`

`$ /proc /cpuinfo`

`[Sample CPU info output truncated for brevity]`

`$ exit`